

LLM4Rec Homework Report — Assignment A02

Student: Anna Grishkina

Team: Individual

Date: 2026-09-22

Assignment: A02 — Content-Based Movie Recommender (Week 2)

Live app: <https://af851574-oss.github.io/RecSys-LLMs/week2/>

Abstract

The starter content-based recommender compared movies with the Jaccard index over genre sets, accepted a single movie, and returned only two recommendations. I replaced the matching with cosine similarity over binary genre vectors, added a profile mode that averages the vectors of up to three watched movies, and extended the output to a scored Top-5 list. A plain-Node regression test parses the real MovieLens 100K files, checks the cosine identities, and verifies one hand-computed pair: Star Wars vs. The Empire Strikes Back equals $5/\sqrt{30} = 0.912871$. The app works locally and on GitHub Pages.

Index Terms — content-based filtering, cosine similarity, user profile, MovieLens 100K, JavaScript

1. Introduction

Problem statement. The Week 2 starter app recommends similar movies from the MovieLens 100K catalog [1], [2]. It scores candidates with the Jaccard index on genre sets, takes the top two, and supports exactly one liked movie.

Motivation. Jaccard is an unweighted binary overlap: it ignores how many genre flags each movie carries and cannot fold several watched movies into one taste vector. The course assignment asks for cosine similarity and profile-based Top-5 recommendations.

Concrete example. Star Wars (1977) carries five genre flags in `u.item` and The Empire Strikes Back (1980) carries six, sharing five. Jaccard reports $5/6 = 0.833$, which penalizes the pair for the extra Drama flag; cosine reports $5/(\sqrt{5} \cdot \sqrt{6}) = 0.9129$, reflecting the shared mix rather than the difference in size.

Contributions.

- Replaced the Jaccard formula with cosine similarity over the genre vectors, with scores shown in the UI.
- Added the profile mode: one pick stays item-to-item, two or three picks build a taste vector by averaging and recommend Top-5.
- Added a dependency-free regression test that parses the real data files and verifies a hand-computed similarity value.
- Compared the two modes on a concrete example and analyzed the normalization effect.

2. Related Work

The baseline is the instructor's Week 2 implementation, which parses `u.item` and `u.data` in the browser and ranks with Jaccard [1]. The data and its column schema come from the MovieLens 100K dataset [2], [3]. Cosine similarity is the standard content-based measure that compares the angle between feature vectors rather than their raw overlap [4]; the Jaccard index it replaces is the set-intersection baseline [5]. I considered TF-IDF over titles, but the catalog ships structured genre flags and no plot text, so binary genre vectors are the honest representation. Rating-weighted profiles were another option, but the UI flow starts from watched titles, not rated ones, so every pick contributes weight $1/n$.

3. Method

Approach. Each movie is an 18-dimensional binary vector over the genre flags of `u.item` (the file's 19th column set, including the leading placeholder, maps onto 18 names as in the starter parser). The similarity between vectors A and B is $\cos(A, B) = A \cdot B / (|A| \cdot |B|)$. With two or three picks, the profile vector averages their vectors dimension-wise, so genres shared by the picks keep the most weight.

Pipeline. Multi-select (capped at 3) → genre vectors → single vector or averaged profile → cosine against every candidate movie → sort descending, ties by movie id → Top-5 list with scores, excluding the picks, labeled with the active mode.

Tools and libraries.

- HTML, CSS, and browser JavaScript (no framework, no build step)
- Node.js v24.10.0 for the regression check (`node:vm` sandbox over the real parser)
- Python 3 `http.server` for local serving
- Google Chrome 152.0.7977.83 headless for the DOM check
- GitHub Pages for static hosting

Key decision. I replaced the Jaccard formula entirely instead of patching it, and kept `data.js` untouched. While reading the parser I noticed its genre names are shifted by one column relative to the 19 flag columns (so the Western column is

never read); because the name→vector round trip is consistent for every movie, this is a dimension permutation that does not change cosine scores. Per the assignment scope I documented it rather than silently rewriting the data layer.

Configuration. Top-k = 5, at most 3 picks, stable tie-breaking by movie id. MovieLens 100K: 1682 movies, 100000 ratings. No training, dataset split, or random seed is involved.

4. Experiments

Setup. I first confirmed the baseline loads 1682 movies and 100000 ratings by running the real parser under Node, then changed the scoring and UI, then reran all checks.

Check	Baseline	Fixed version
Similarity formula	Jaccard	Cosine
Input movies	1	1–3
Output	Top-2, no scores	Top-5 with scores and mode
Profile vector	none	average of picks
Regression test	none	week2/test.mjs, all assertions

Verification performed.

1. node week2/test.mjs — PASS: 1682 movies, 100000 ratings, $\text{cosine}(v,v) \approx 1$, disjoint vectors ≈ 0 , hand-computed $\text{cosine}(50, 172) = 0.912871$, valid 3-movie Top-5, Jaccard gone from script.js.
2. node --check on week2/script.js and week2/data.js — both exit 0.
3. Local GET /week2/ over python3 -m http.server — HTTP 200.
4. Headless Chrome --dump-dom on the served page rendered the loaded state with 1683 dropdown options (placeholder + 1682 movies). Button clicks are not drivable with --dump-dom alone, so the Top-5 lists below come from the real extracted core functions under Node — the same code path the button triggers.
5. I re-derived both Top-5 lists independently from the raw u.item flags with a separate script; the results matched exactly.

Hand-computed example. From the raw flags: Star Wars = {Action, Adventure, Romance, Sci-Fi, War}, Empire Strikes Back adds Drama. Shared genres = 5, so $\text{cosine} = 5/(\sqrt{5} \cdot \sqrt{6}) = 5/\sqrt{30} = 0.912871$. The test hardcodes this value to six decimals and the implementation matches.

Top-5 for Star Wars (1977), item-to-item:

id	title	score
181	Return of the Jedi (1983)	1.000000
172	Empire Strikes Back, The (1980)	0.912871
271	Starship Troopers (1997)	0.894427
498	African Queen, The (1951)	0.894427
62	Stargate (1994)	0.774597

Top-5 for the profile [Star Wars, Empire Strikes Back, GoodFellas]:

id	title	score
181	Return of the Jedi (1983)	0.894427
271	Starship Troopers (1997)	0.800000
498	African Queen, The (1951)	0.800000
631	Crying Game, The (1992)	0.800000
720	First Knight (1995)	0.800000

5. Discussion

Failure case. A movie with no genre flags yields a zero vector; the implementation returns similarity 0 for it instead of dividing by zero. Genre-poor movies rank low under cosine because their short vectors share little with anything.

Root cause. Jaccard only sees $|\text{intersection}|/|\text{union}|$ of two sets: it cannot express “how much of this movie is like the query”, it ignores vector length, and it accepts exactly two arguments, which blocks profile aggregation.

Item-to-item vs. profile. With one pick, Return of the Jedi scores a perfect 1.000000 (identical flags to Star Wars) and the list is pure space opera. With the three-movie profile, the leader stays but drops to 0.894427, Stargate falls out, and Drama-leaning titles (The Crying Game, First Knight) enter at 0.800000 — the average vector rewards genres that more picks share, so GoodFellas’ Crime/Drama pulls the mix toward its neighborhood.

Bias mitigation. Cosine divides the shared-genre count by both vector lengths, so every movie competes at unit length: a multi-genre “blockbuster” vector cannot flood the ranking by touching many genres, and every score is bounded by 1. Under raw overlap such movies would win against everyone; under cosine they only win against movies whose mix they genuinely match.

Catalog discovery. The single-item list keeps recommending near-clones, which feeds fatigue. The profile list surfaces titles that match the shared core but differ in the rest (The African Queen, 1951, appears in both; The Crying Game only in the profile list), which is the long-tail behavior that supports retention.

What worked.

- Extracting the pure functions into a marked CORE block made the browser code testable under Node without a framework.
- Averaging picked vectors was sufficient to shift the Top-5 meaningfully — no learned model needed.

What surprised me or did not work.

- Two different movies can score exactly 1.000000 (Star Wars and Return of the Jedi have identical flag sets), so I kept the id tie-break for a stable order.
- The starter’s genre-name shift (Section 3) does not affect cosine at all — a permutation of dimensions changes neither dot product nor norms.
- Jaccard and cosine agreed on the disjoint pair (Toy Story vs. GoodFellas = 0 in both), so the metrics only visibly diverge on partial overlap.

Next improvement. Fix the parser’s genre-name alignment and include the unused 19th column, weight profile picks by the user’s actual ratings, and add title text features. I did not do this now because the assignment scope is the similarity and profile logic, and the current numbers are reproducible and consistent.

6. AI Usage Disclosure

AI tools used. Pi Coding Agent 0.85.1 with an OpenAI model through OpenRouter.

How I used AI. I used it to inspect the starter, organize the five-loop prompts, implement the cosine and profile logic, prepare the test, and draft the report.

What I verified myself. I recomputed $5/\sqrt{30}$ by hand, reran the regression test and both syntax checks, re-derived the two Top-5 lists from the raw `u.item` flags with a separate script, opened every reference, and inspected the final PDF and the published page.

What I trusted without verification. I trusted that the genre flags in `u.item` describe the movies correctly; I did not cross-check individual movies against external databases.

Session log. The attached `argrishkina_a02_session.json` is a privacy-redacted audit generated from the task-only implementation session and Git history. It contains verification tool calls, file changes, and milestones, but excludes conversational content and replaces local account identifiers. It is not an unmodified Pi export. The prompts and investigation notes are also recorded in `week2/prompt.md`.

References

- [1] J. S. Jin, “Content-Based Movie Recommender source,” *GitHub*, 2025. [Online]. Available: <https://github.com/dryjins/RecSys-LLMs/blob/main/week2/script.js>. [Accessed: 2026-09-22].
- [2] GroupLens Research, “MovieLens 100K Dataset.” [Online]. Available: <https://grouplens.org/datasets/movielens/100k/>. [Accessed: 2026-09-22].
- [3] GroupLens Research, “ml-100k README (u.item genre column schema).” [Online]. Available: <https://files.grouplens.org/datasets/movielens/ml-100k-README.txt>. [Accessed: 2026-09-22].
- [4] scikit-learn developers, “sklearn.metrics.pairwise.cosine_similarity.” [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html. [Accessed: 2026-09-22].
- [5] Wikipedia, “Jaccard index.” [Online]. Available: https://en.wikipedia.org/wiki/Jaccard_index. [Accessed: 2026-09-22].